

DAY-5 ASSIGNMENT [DSA-BS]

UDAY CHIRRA

2303A510G2

11. Binary Search

```
class Solution {
    public int search(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) {
                return mid;
            }

            if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return -1;
    }
}
```

12. Search Insert Position

```
class Solution {
    public int searchInsert(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) {
                return mid;
            }
        }
    }
}
```

```

        if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return left;
}

```

13. First Bad Version

```

public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int left = 1;
        int right = n;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (isBadVersion(mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }
}

```

14. Sqrt(x)

```

class Solution {
    public int mySqrt(int x) {
        if (x < 2) {
            return x;
        }
    }
}

```

```

int left = 1;
int right = x / 2;

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (mid <= x / mid) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

return right;
}
}

```

15. Find First and Last Position of Element

```

class Solution {
    public int[] searchRange(int[] nums, int target) {
        int first = findFirst(nums, target);
        int last = findLast(nums, target);

        return new int[]{first, last};
    }

    private int findFirst(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;
        int answer = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) {
                answer = mid;
                right = mid - 1;
            } else if (nums[mid] < target) {

```

```

        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

return answer;
}

private int findLast(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;
    int answer = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            answer = mid;
            left = mid + 1;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return answer;
}
}

```

16. Search in Rotated Sorted Array

```

class Solution {
    public int search(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

```

```

    if (nums[mid] == target) {
        return mid;
    }

    if (nums[left] <= nums[mid]) {
        if (nums[left] <= target && target < nums[mid]) {
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    } else {
        if (nums[mid] < target && target <= nums[right]) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
}

return -1;
}
}

```

17. Find Peak Element

```

class Solution {
    public int findPeakElement(int[] nums) {
        int left = 0;
        int right = nums.length - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] > nums[mid + 1]) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }
}

```

```
    }  
}
```

18. Search a 2D Matrix

```
class Solution {  
    public boolean searchMatrix(int[][] matrix, int target) {  
        int rows = matrix.length;  
        int cols = matrix[0].length;  
  
        int left = 0;  
        int right = rows * cols - 1;  
  
        while (left <= right) {  
            int mid = left + (right - left) / 2;  
  
            int value = matrix[mid / cols][mid % cols];  
  
            if (value == target) {  
                return true;  
            }  
  
            if (value < target) {  
                left = mid + 1;  
            } else {  
                right = mid - 1;  
            }  
        }  
  
        return false;  
    }  
}
```

19. Koko Eating Bananas

```
class Solution {  
    public int minEatingSpeed(int[] piles, int h) {  
        int left = 1;  
        int right = 0;  
  
        for (int pile : piles) {  
            right = Math.max(right, pile);  
        }  
    }  
}
```

```

    }

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (canFinish(piles, h, mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }

    return left;
}

private boolean canFinish(int[] piles, int h, int speed) {
    long hours = 0;

    for (int pile : piles) {
        hours += (pile + (long) speed - 1) / speed;

        if (hours > h) {
            return false;
        }
    }

    return true;
}
}

```